项目 3 基础模块使用

1、 预备知识

Go2 机器人的基础模块涵盖了 DDS 通信、避障、音量和灯光控制等多个接口，这些模块构成了后续应用开发的基础，为实现各种功能提供了强大的支持。通过这些接口，开发者可以轻松地为 Go2 机器人定制和扩展各种应用，满足不同的使用需求。

1.1、 DDS 通信接口

DDS(全称 Data Distribution Service 数据分发服务)，是一个中间件，由 OMG 发布的分布式通信规范，采用发布/订阅模型，提供多种 QoS 服务质量策略，以保障数据进行实时、高效、灵活地分发，可满足各种分布式实时通信应用需求。

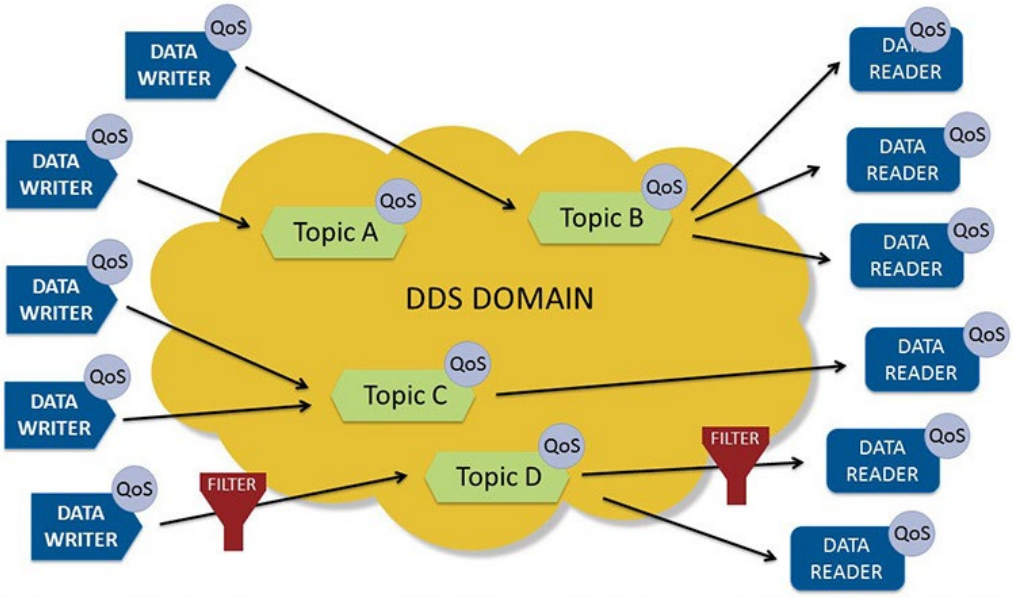


图 1-1 DDS 协议架构图

unitree_sdk2 在 DDS 上做了一层封装，支持 DDS 组件的 Qos 配置，对应用开发提供简单的封装接口，并实现了基于 DDS Topic 的 RPC 机制,适用于在 Go2 内部进程间以及 Go2 外部与内部的进程间通过发布/订阅和请求/响应两种方式完成不同场景下的数据通信。

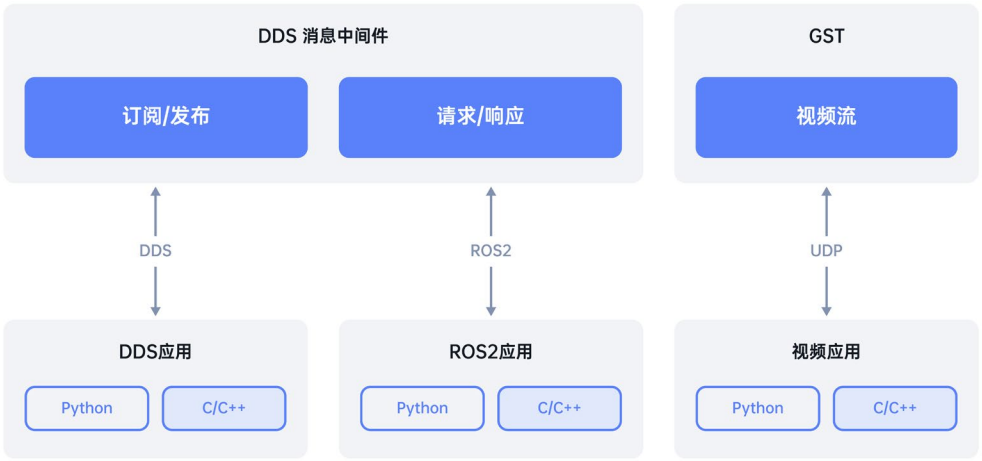


图 1-2 DDS 消息中间件

1.2、 避障接口

避障接口通过 RPC 方式实现对 Go2 避障功能的开关和开关状态获取。

1.3、 音量/灯光接口

音量灯光接口通过 RPC 方式实现对 Go2 音量和灯光的控制和

信息获取等功能。

1.4、 设备状态接口

设备状态接口通过 RPC 方式实现对 Go2 内部的服务控制、获取服务状态、设备状态和系统资源使用信息等功能。

2、 DDS 通信接口应用

2.1、 DDS 通信的 C++例程

2.1.1、 发布端代码实现

2.1.1.1、 流程图及实现思路

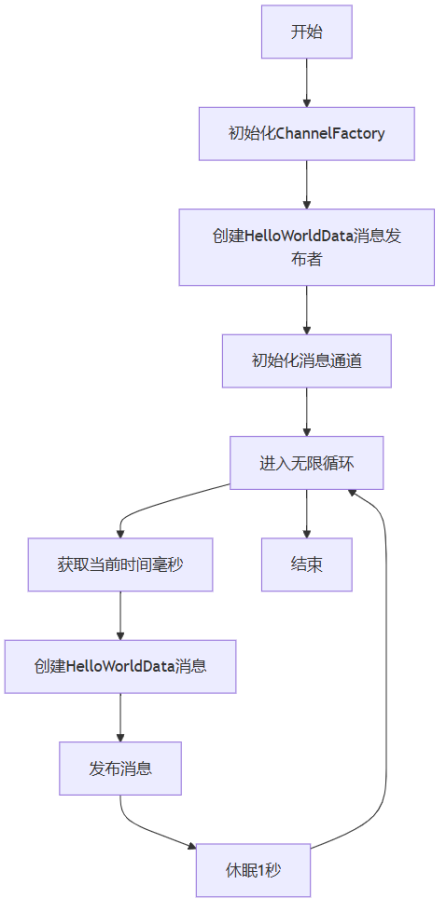


图 2-1 发布端程序流程图

程序流程说明如下：

①初始化工作：

调用 ChannelFactory::Instance()->Init(0)初始化通道工厂，这里的参数 0 表示使用默认配置。

创建一个ChannelPublisher<HelloWorldData::Msg>对象，并使用宏定义的 TOPIC 作为主题。

②初始化消息通道：

调用 publisher.InitChannel();初始化通道，以便可以发布消息。

③进入循环处理：

程序进入一个无限循环，在这个循环中，程序将不断发布消息。

获取当前时间毫秒，使用获取到的时间和固定的字符串"HelloWorld."创建一个 HelloWorldData::Msg 类型的消息。

调用 publisher.Write(msg)将消息发布到通道，使程序休眠 1 秒，然后循环执行。

2.1.1.2、 完整代码

发布 Topic(主题)为**“TopicHelloWorld”**，可修改为自定义主题。

```
#include <unitree/robot/channel/channel_publisher.hpp>
#include <unitree/common/time/time_tool.hpp>
#include "HelloWorldData.hpp"
#define TOPIC "TopicHelloWorld"
using namespace unitree::robot;
using namespace unitree::common;
int main()
{
    ChannelFactory::Instance()->Init(0);
    ChannelPublisher<HelloWorldData::Msg> publisher(TOPIC);
    publisher.InitChannel();
    while (true)
    {
        HelloWorldData::Msg
msg(unitree::common::GetCurrentTimeMillisecond(), "HelloWorld.");
        publisher.Write(msg);
        sleep(1);
    }
```

```
        return 0;

    }
```

2.1.2、 订阅端代码实现

2.1.2.1、 流程图及实现思路

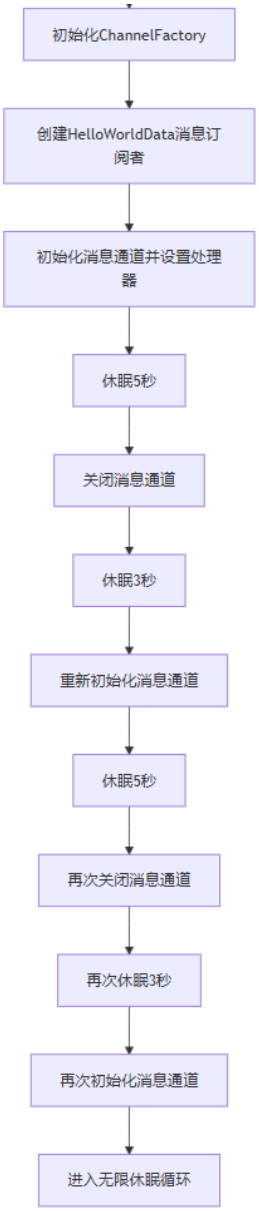


图 2-1 订阅端程序流程图

①初始化工作：

调用 ChannelFactory::Instance()->Init(0)初始化通道工厂，参数 0 表示使用默认配置。

创建一个 ChannelSubscriber<HelloWorldData::Msg>对象，并使用宏定义的 TOPIC 作为主题。

调用 subscriber.InitChannel(Handler)初始化通道，并设置消息处理函数 Handler。

②消息等待与接收

使程序休眠 5 秒，等待可能的消息；

调用 subscriber.CloseChannel()关闭消息通道。

休眠 3 秒后，再次调用 subscriber.InitChannel()初始化消息通道，再次休眠 5 秒，再次调用 subscriber.CloseChannel()关闭消息通道；重复以上过程。

③进入无限休眠循环

程序进入一个无限循环，在这个循环中，程序将不断休眠 10 秒。

2.1.2.2、 完整代码

订阅 Topic(主题)为**“TopicHelloWorld”**，可修改为自定义主题。

```
#include <unitree/robot/channel/channel_subscriber.hpp>
```

```
#include <unitree/common/time/time_tool.hpp>
#include "HelloWorldData.hpp"
#define TOPIC "TopicHelloWorld"
using namespace unitree::robot;
using namespace unitree::common;
void Handler(const void* msg)
    const HelloWorldData::Msg* pm = (const HelloWorldData::Msg*)msg;
    std::cout << "userID:" << pm->userID() << ", message:" << pm->message() << std::endl;
}
int main()
{
    ChannelFactory::Instance()->Init(0);
    ChannelSubscriber<HelloWorldData::Msg> subscriber(TOPIC);
    subscriber.InitChannel(Handler);
    sleep(5);
    subscriber.CloseChannel();
```

```
            std::cout << "reseted. sleep 3" << std::endl;
            sleep(3);
            subscriber.InitChannel();
            sleep(5);
            subscriber.CloseChannel();
            std::cout << "reseted. sleep 3" << std::endl;
            sleep(3);
            subscriber.InitChannel();
            while (true)
            {
                sleep(10);
            }
            return 0;
        }
            while (true)
            {
                HelloWorldData::Msg
msg(unitree::common::GetCurrentTimeMillisecond(), "HelloWorld.");
```

```
            publisher.Write(msg);
            sleep(1);
        }
        return 0;
    }
```

2.1.3、 数据类代码实现

以下代码是在应用中为 HelloWorldData::Msg 数据类型注册类型属性，代码中定义了一个模板特化，用于获取 HelloWorldData::Msg 类型的属性向量（propvec）。这个向量包含了数据类型中每个成员的属性，这些属性将被用于 DDS 中的序列化、反序列化和数据传输。

```
#include "HelloWorldData.hpp"
namespace org{
namespace eclipse{
namespace cyclonedds{
namespace core{
namespace cdr{
template<>
```

```
propvec &get_type_props<::HelloWorldData::Msg>() {
    static thread_local std::mutex mtx;
    static thread_local propvec props;
    static thread_local entity_properties_t *props_end = nullptr;
    static thread_local std::atomic_bool initialized {false};
    key_endpoint keylist;
    if (initialized.load(std::memory_order_relaxed)) {
        auto ptr = props.data();
        while (ptr < props_end)
            (ptr++)->is_present = false;
        return props;
    }
    std::lock_guard<std::mutex> lock(mtx);
    if (initialized.load(std::memory_order_relaxed)) {
        auto ptr = props.data();
        while (ptr < props_end)
            (ptr++)->is_present = false;
        return props;
```

```
        }

        props.clear();

        props.push_back(entity_properties_t(0,    0,    false,    bb_unset,
extensibility::ext_final));   //root

        props.push_back(entity_properties_t(1,              0,              false,
get_bit_bound<int64_t>(), extensibility::ext_final, false));   //::userID

        props.push_back(entity_properties_t(1,    1,    false,    bb_unset,
extensibility::ext_final, false));   //::message

        entity_properties_t::finish(props, keylist);

        props_end = props.data() + props.size();

        initialized.store(true, std::memory_order_release);

        return props;

    }

    } //namespace cdr

    } //namespace core

    } //namespace cyclonedds

    } //namespace eclipse

    } //namespace org
```

2.2、 DDS 通信的 Python 例程

2.2.1、 发布端代码实现

```
import time
from unitree_sdk2py.core.channel import ChannelPublisher, ChannelFactoryInitialize
from user_data import *
if __name__ == "__main__":
    ChannelFactoryInitialize()
    # Create a publisher to publish the data defined in UserData class
    pub = ChannelPublisher("topic", UserData)
    pub.Init()
    for i in range(30):
        # Create a Userdata message
        msg = UserData(" ", 0)
        msg.string_data = "Hello world"
        msg.float_data = time.time()
```

```
            # Publish message
            if pub.Write(msg, 0.5):
                print("Publish success. msg:", msg)
            else:
                print("Waitting for subscriber.")
            time.sleep(1)
        pub.Close()
```

2.2.2、 订阅端代码实现

```
    import time
    from unitree_sdk2py.core.channel import ChannelSubscriber,
ChannelFactoryInitialize
    from user_data import *
    if __name__ == "__main__":
        ChannelFactoryInitialize()
        # Create a subscriber to subscribe the data defined in UserData
class
        sub = ChannelSubscriber("topic", UserData)
        sub.Init()
```

```
        while True:
            msg = sub.Read()
            if msg is not None:
                print("Subscribe success. msg:", msg)
            else:
                print("No data subscribed.")
                break
        sub.Close()
```

2.2.3、 数据类代码实现

```
    rom dataclasses import dataclass
    from cyclonedds.idl import IdlStruct
    # This class defines user data consisting of a float data and a string
data
    @dataclass
    class UserData(IdlStruct, typename="UserData"):
        string_data: str
        float_data: float
```

2.3、 项目测试(Python)

在开发主机上分别运行发布端和订阅端代码，测试项目的运行情况，正常运行的情况是订阅端持续收到发布端的消息。

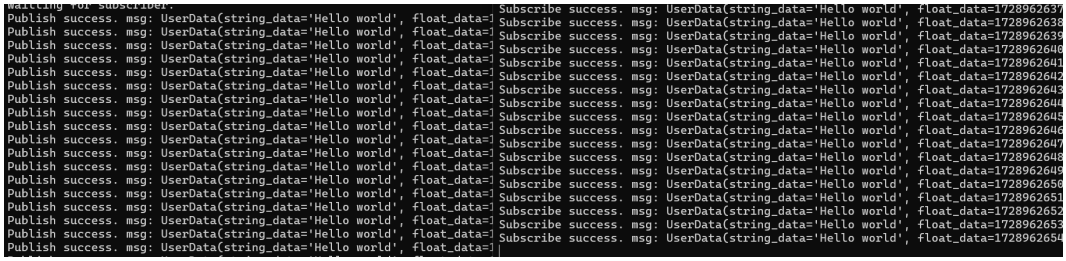


图 2-2 DDS 协议测试

3、 避障接口应用

3.1、 避障服务接口 ObstaclesAvoidClient 类

ObstaclesAvoidClient 是 Go2 避障服务提供的 Client，通过 ObstaclesAvoidClient 可以通过 RPC 方式实现对 Go2 避障功能的开关和开关状态获取，具体类与接口描述如下

类名	构造与析构
ObstaclesAvoidClient	explicit ObstaclesAvoidClient(); ~ObstaclesAvoidClient();

函数名	Set
函数原型	int32_t Set(bool enable)
功能概述	设置避障功能开启或关闭。
参数	**enable**：设置 true 为开启，false 为关闭。
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Get
函数原型	int32_t Get(bool& enable);
功能概述	获取避障功能是否开启。
参数	**enable**：返回值， true 为开启，false 为关闭。
返回值	调用成功返回 0，否则返回相关错误码。

函数名	UseRemoteCommandFromApi
函数原型	int32_t UseRemoteCommandFromApi(bool is_remote_commands_from_api)
功能概述	抢夺遥控器的速度指令控制权。
参数	**is_remote_commands_from_api**：设置 true 为开启抢夺，false 为关闭抢夺。
返回值	调用成功返回 0，否则返回相关错误码。
备注	需将is_remote_commands_from_api设为true，才能通过api控制机器狗避障

函数名	Move
函数原型	int32_t Move(float x, float y, float yaw);
功能概述	避障模式下的移动控制。
参数	**x**：机身x方向速度指令，限制为-1.5~+1.5(m/s)； **y**：机身y方向速度指令，限制为-1~+1(m/s)； **yaw**：机身yaw角速度指令，限制为-1.57~+1.57(rad/s)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	需将is_remote_commands_from_api设为true，才能通过Move函数控制机器狗避障

3.2、 流程图及实现思路

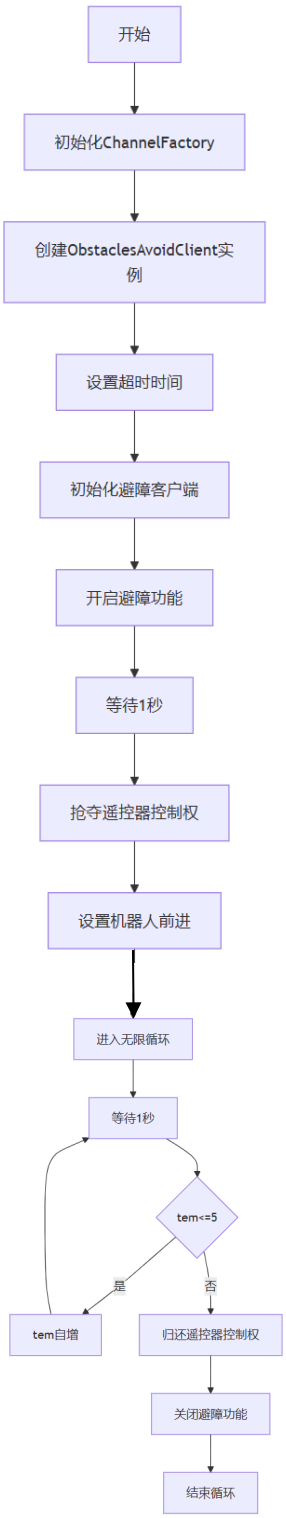


图 3-1 避障接口应用流程图

①初始化工作：

初始化通道工厂对象并指定网络接口地址；创建一个

ObstaclesAvoidClient 对象 sc。

②初始化避障客户端：调用 sc.Init()初始化避障客户端，调用 sc.SwitchSet(true)开启避障功能。

③抢夺遥控器控制权：

调用 sc.UseRemoteCommandFromApi(true)抢夺遥控器的速度控制权。

④设置机器人前进：

调用 sc.Move(1.0,0.0,0.0)设置机器人以 1m/s 的速度沿 x 方向前进。

⑤进入无限循环：

程序进入一个无限循环，在这个循环中，程序将不断检查时间并控制避障功能。等待 1 秒，检查变量 tem 的值是否小于或等于 5；如果是，tem 自增；如果否，归还遥控器控制权并关闭避障功能。

⑥归还遥控器控制权：

调用 sc.UseRemoteCommandFromApi(false)归还遥控器控制权。

3.3、 避障应用的 C++例程

```
#include
```

```
<unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp>

using namespace unitree::common;

using namespace unitree::robot;

using namespace unitree::robot::go2;

int main()

{

    unitree::robot::ChannelFactory::Instance()->Init(0, "enp0s3");

    // enp0s3 为网口号，用户根据自身情况修改

    unitree::robot::go2::ObstaclesAvoidClient sc;

    sc.SetTimeout(5.0f);

    sc.Init();

    sc.SwitchSet(true);//开启避障

    usleep(1000000);

    sc.UseRemoteCommandFromApi(true);//抢夺遥控器的速度控制权

    sc.Move(1.0,0.0,0.0);//沿着机体 x 方向以 1m/s 的速度运动，碰到障碍
物自动避障

    int tem = 0;

    while (true)
```

```
    {
        usleep(1000000);
        if(tem<=5)
        {
            tem++;
        }
        else
        {
            sc.UseRemoteCommandFromApi(false);//5 秒后关闭
            sc.SwitchSet(false);//5 秒后关闭
        }
    }
    return 0;
}
```

3.4、 避障接口应用的 Python 例程

3.4.1、 初始化避障服务对象

```
        client = ObstaclesAvoidClient()
        client.SetTimeout(3.0)
```

```
client.Init()
```

3.4.2、 关闭避障服务开关

```
client.SwitchSet(False)
time.sleep(2)
```

3.4.3、 开启避障服务开关

```
client.SwitchSet(True)
time.sleep(2)
```

3.4.4、 完整例程

```
import time
import sys
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.obstacles_avoid.obstacles_avoid_client import ObstaclesAvoidClient
if __name__ == "__main__":
    if len(sys.argv)>1:
        ChannelFactoryInitialize(0, sys.argv[1])
```

```
        else:
            ChannelFactoryInitialize(0)
        client = ObstaclesAvoidClient()
        client.SetTimeout(3.0)
        client.Init()
        client.SwitchSet(False)
        time.sleep(2)
        client.SwitchSet(True)
        time.sleep(2)
        client.UseRemoteCommandFromApi(True)
        time.sleep(2)
        while True:
            client.Move(0.5, 0.0, 0.0)
            time.sleep(0.02)
```

3.5、 项目测试

运行代码，测试项目运行情况。一是查看 APP 中避障服务状态是否产生相应变化；二是用遥控器控制 Go2 机器人运动，进行避障

测试。



图 3-2 服务状态图

4、 灯光/亮度接口应用

VuiClient 是语音和灯光服务状态服务提供的 Client，通过 VuiClient 可以通过 RPC 方式实现对 Go2 音量和灯光的控制和信息获取等功能。

SDK 目前提供了设置/获取**白色**灯光两种方法，开发者可以根据不同的场景需求，自定义 LED 的亮度。调节档位为 0 到 10 之间。

4.1、 类和接口说明

类名	**构造与析构**
VuiClient	explicit VuiClient(); ~VuiClient();
函数名	**SetBrightness**
函数原型	int32_t SetBrightness(int level)
功能概述	设置灯光亮度。
参数	**level**：灯光亮度，取值范围 0~10。

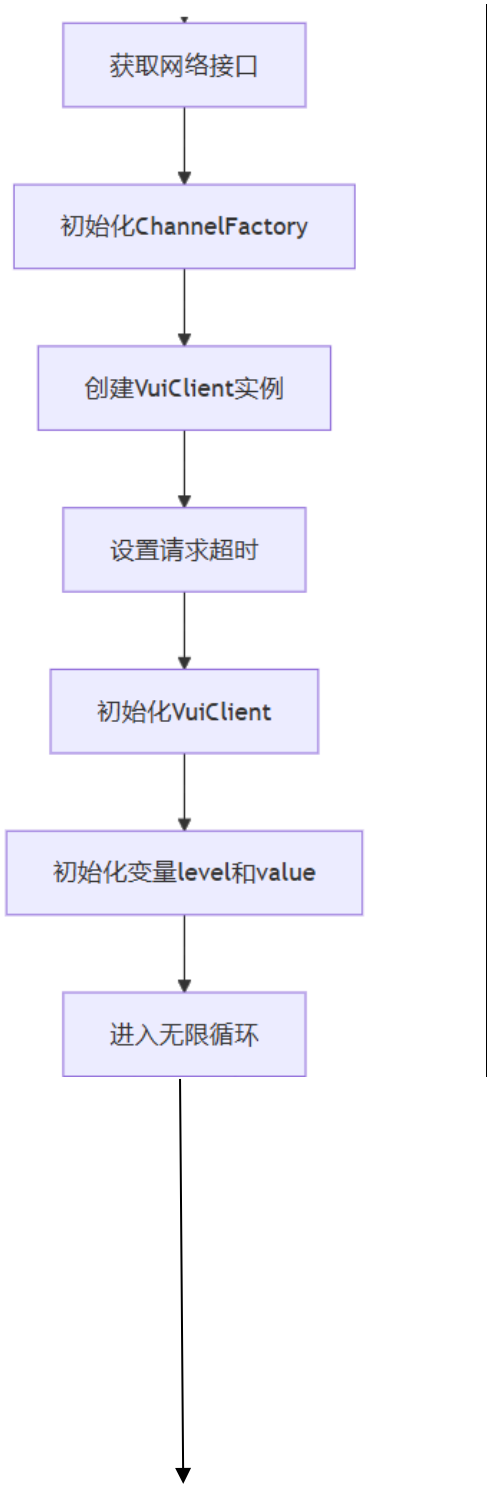
返回值	调用成功返回 0，否则返回相关错误码。
函数名	**GetBrightness**
函数原型	int32_t GetBrightness(int& level)
功能概述	获取灯光亮度。
参数	**level**：返回的灯光亮度。
返回值	调用成功返回 0，否则返回相关错误码。

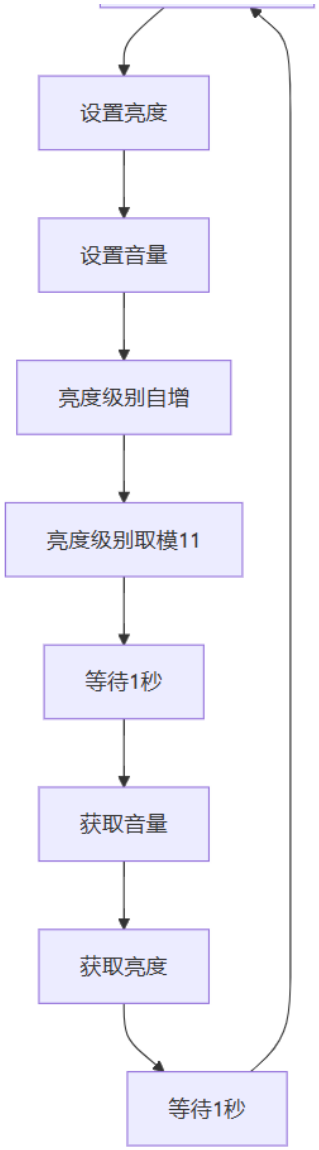
表 4-1 亮度控制接口

函数名	**SetVolume**
函数原型	int32_t SetVolume(int level)
功能概述	设置音量强度。
参数	**level**：音量强度，取值范围 0~10。
返回值	调用成功返回 0，否则返回相关错误码。
函数名	**GetVolume**
函数原型	int32_t GetVolume(int& level)
功能概述	获取音量强度。
参数	**level**：返回的音量强度。
返回值	调用成功返回 0，否则返回相关错误码。

表 4-2 音量控制接口

4.2、 C++例程实现


获取网络接口
初始化ChannelFactory
创建VuiClient实例
设置请求超时
初始化VuiClient
初始化变量level和value
进入无限循环



①初始化 ChannelFactory：

使用获取的网络接口名称初始化 ChannelFactory。

②创建 VuiClient 实例：

创建 VuiClient 类的实例。

③设置请求超时：

设置 VuiClient 的请求超时时间为 1.0 秒。

④初始化 VuiClient：

调用 VuiClient 的 Init 方法进行初始化。

⑤初始化变量 level 和 value：

初始化用于控制亮度和音量的变量 level，以及用于存储获取到的音量和亮度值的变量 value。

⑥进入无限循环：

进入一个无限循环，循环体内包含对机器人音量和亮度的控制。

⑦设置亮度和音量：

调用 VuiClient 的 SetBrightness 方法设置亮，调用 VuiClient 的 SetVolume 方法设置音量。

⑨亮度级别自增：

亮度级别自增，亮度级别取模 11：亮度级别取模 11，以确保 level 的值在 0 到 10 之间循环。

⑩获取音量和亮度：

调用 VuiClient 的 GetVolume 方法获取当前音量，调用 VuiClient 的 GetBrightness 方法获取当前亮度。

```
#include <unitree/robot/go2/vui/vui_client.hpp>
```

```
    int main(int32_t argc, const char** argv)
    {
        if (argc < 2)
        {
            std::cout << "Please fill in the correct network card name"
<< std::endl;
            exit(0);
        }
        /*
         * Initilaize ChannelFactory
         */
        std::string networkInterface = "eth0";
        networkInterface = argv[1];
        unitree::robot::ChannelFactory::Instance()->Init(0,
networkInterface);
        unitree::robot::go2::VuiClient vc;
        /*
         * Set request timeout 1.0s
```

```
            */
            vc.SetTimeout(1.0f);
            vc.Init();
            //Test Api
            int level = 0, value = 0;
            int ret;
            while (true)
            {
                ret = vc.SetBrightness(level);
                std::cout << "SetBrightness  level=" << level << ", api
return:" << ret << std::endl;
                ++level %= 11;
                sleep(1);
                ret = vc.GetBrightness(value);
                std::cout << "GetBrightness value=" << value << ", api
return:" << ret << std::endl;
                sleep(1);
            }
```

```
    return 0;
}
```

4.3、 Python 例程实现

```
import time
import sys
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.vui.vui_client import VuiClient
if __name__ == "__main__":
    if len(sys.argv)>1:
        ChannelFactoryInitialize(0, sys.argv[1])
    else:
        ChannelFactoryInitialize(0)
    client = VuiClient()
    client.SetTimeout(3.0)
    client.Init()
    for i in range(1, 11):
        code, level = client.GetBrightness()
```

```
                if code != 0:
                    print("get brightness error. code:", code)
                else:
                    print("get brightness success. level:", level)
                time.sleep(1)
print("##################SetBrightness####################"
)
                code = client.SetBrightness(i)
                if code != 0:
                    print("set brightness error. code:", code)
                else:
                    print("set brightness success. level:", i)
                time.sleep(1)
            print("##################SetBrightness
0####################")
            code    = client.SetBrightness(0)
            if code != 0:
                print("set brightness error. code:", code)
```

```
            else:
                print("set brightness 0 success.")
            for i in range(1, 11):


print("##################GetVolume####################")
                code, level = client.GetVolume()
                if code != 0:
                    print("get volume error. code:", code)
                else:
                    print("get volume success. level:", level)
                time.sleep(1)
print("##################SetVolume####################")
                code = client.SetVolume(i)
                if code != 0:
                    print("set volume error. code:", code)
                else:
                    print("set volume success. level:", i)
                time.sleep(1)
```

```
        print("##################SetVolume
0####################")
        code    = client.SetVolume(0)
        if code != 0:
            print("set volume error. code:", code)
        else:
            print("set volume 0 success.")
```

4.4、 项目测试

运行代码，测试项目运行情况，观察 Go2 机器人头部的音量和光照强度是否在循环变化。

5、 项目总结

在本项目中，我们深入学习并掌握了 Go2 机器人 SDK 内含的多个基础模块，包括 DDS 通信、避障、音量和灯光控制等。这些技能的积累将为我们即将开展的高级运动控制项目提供坚实的基础，使我们能够更高效地开发和实现更复杂的机器人应用。